

Technical Appendix

Code #1 Overview

The provided code contains functionalities to assess the strength and security of passwords. It comprises two primary functions:

`download_common_passwords(url):`

- This function employs the requests library to perform an HTTP GET request to a specified URL.
- Upon successful retrieval of the file, it saves the content as `common_passwords.txt` in the project directory.
- The function checks for the file's presence before initiating the download to avoid redundant downloads.

`check_password(thepassword):`

- This function evaluates the strength and integrity of a given password against specific predefined criteria.
- Utilizing the re library, it implements regular expressions to analyze the password characteristics, including:
 - Length validation (minimum length requirement).
 - Presence of digits, uppercase and lowercase letters, symbols, and repeating patterns within the password.
- The function cross-references the input password against a local file, `common_passwords.txt`, which contains a list of known common passwords.
- Based on the checks performed, it computes an overall score, providing an indication of the password's strength and resistance to common vulnerabilities.

External Libraries

The code makes use of the following external libraries:

- re (Regular Expressions):
 - Utilized for pattern matching and analysis of various password characteristics.
 - Regular expressions validate minimum length, presence of digits, uppercase and lowercase letters, symbols, and repeating patterns within passwords.
- requests (HTTP Requests):
 - Employed for performing HTTP GET requests to fetch external resources.
 - Utilized specifically to retrieve a file containing common passwords from a remote server.

Functions and Functionalities

- `download_common_passwords(url):`
 - Initiates an HTTP GET request using the requests library to fetch a file from the specified URL.
 - Upon successful retrieval, writes the content to a local file, `common_passwords.txt`, in the project directory.
 - Verifies the existence of the file before initiating the download process to prevent unnecessary redundant downloads.
- `check_password(thepassword):`
 - Accepts a password input and assesses its strength based on predefined criteria.
 - Utilizes regular expressions from the re library to validate multiple password characteristics:
 - Minimum length requirement.
 - Presence of digits, uppercase and lowercase letters, symbols, and repeating patterns.
 - Compares the input password against a local file, `common_passwords.txt`, to determine if it matches known common passwords.

- Computes an overall score, providing an indication of the password's strength relative to the defined criteria.

Data Handling

- Fetching Common Passwords:
 - The `download_common_passwords` function uses the `requests` library to fetch a file containing a list of common passwords from a specified URL.
 - Upon retrieval, the file content is stored locally as `common_passwords.txt`.
 - The script verifies the existence of the local file to prevent repetitive downloads.
- Password Evaluation:
 - The `check_password` function uses regular expressions to evaluate password characteristics such as length, presence of specific character types, and repeated patterns.
 - Compares the input password against the list of known common passwords stored in `common_passwords.txt` to determine if it's a commonly used password.
 - Computes an overall score based on the predefined criteria to assess the password's strength and vulnerability.

Code #1

```
import re
import requests # Import the requests library to fetch the file content

COMMON_PASSWORDS_URL = 'https://lucidar.me/en/security/files/10000-most-common-passwords.txt'

def download_common_passwords(url):
    response = requests.get(url)
    if response.status_code == 200:
        with open('common_passwords.txt', 'wb') as file:
            file.write(response.content)

def check_password(thepassword):
    pass_length = len(thepassword)
    length_check = pass_length >= 8 # is password 8 characters or more in length?

    digit_count = len(re.findall(r"d", thepassword))
    digit_check = re.search(r"d", thepassword) is not None # does password have at least one digit?

    uppercase_count = len(re.findall(r"[A-Z]", thepassword))
    uppercase_check = re.search(r"[A-Z]", thepassword) is not None # does password have at least one
uppercase character?

    lowercase_count = len(re.findall(r"[a-z]", thepassword))
    lowercase_check = re.search(r"[a-z]", thepassword) is not None # does password have at least one
lowercase character?

    symbol_count = len(re.findall(r"W", thepassword))
    symbol_check = re.search(r"W", thepassword) is not None # does password have at least one
symbol?
```

```

repeat_matches = re.findall(r"(.+?)\1", thepassword)
repeat_count = len(repeat_matches)

# Download common passwords file if not already present
try:
    file = open('common_passwords.txt', 'r')
except FileNotFoundError:
    download_common_passwords(COMMON_PASSWORDS_URL)
    file = open('common_passwords.txt', 'r')

common_passwords = [line.strip() for line in file]
is_common = thepassword.lower() in common_passwords

passes_checks = length_check and digit_check and uppercase_check and lowercase_check and
symbol_check and not is_common and not len(repeat_matches) != 0

length_weight = 1 # add 1 for every character
digit_weight = 1 # add 1 for every digit
symbol_weight = 1 # add 1 for every symbol
repeat_weight = -3 # subtract 3 for every repeat
common_weight = -10 # subtract 5 if password is common

overall_score = sum([
    pass_length * length_weight,
    digit_count * digit_weight,
    symbol_count * symbol_weight,
    repeat_count * repeat_weight,
])

if is_common:
    overall_score += common_weight

return {
    'Password Analyzed': thepassword,
    'Passes All Checks?': passes_checks,
    'At least 8 Length?': length_check,
    'At least 1 Digit?': digit_check,
    'At least 1 Uppercase?': uppercase_check,
    'At least 1 Lowercase?': lowercase_check,
    'At least 1 Symbol?': symbol_check,
    'Repeated Patterns?': len(repeat_matches) != 0,
    'Is Common Password?': is_common,
    'Length of Password': pass_length,
    'Number of Digits': digit_count,
    'Number of Uppercase': uppercase_count,
    'Number of Lowercase': lowercase_count,
    'Number of Symbols': symbol_count,
    'Patterns': repeat_matches,

```

```
'Overall Score': overall_score,  
}
```

```
check_password("password123")
```

Code #2 Overview

The provided code generates random passwords and evaluates their strength based on predefined criteria. It consists of two primary functions:

`generate_random_password(length):`

- Generates a random password of a specified length using characters from the ASCII set (letters, digits, and punctuation).
- Validates the generated password against specific criteria such as the presence of digits, uppercase and lowercase letters, symbols, absence of repeating characters, and exclusion of specific patterns like birth years ('1998', '2001', '2003').
- Counts occurrences of different types of characters (digits, uppercase, lowercase, symbols, repeating characters).

`calculate_password_strength(password_info):`

- Calculates the strength of a generated password based on the number of characters, character types, and absence of repeating patterns.
- Assigns weights to different factors (length, character types) and computes a simple strength score for the password.

External Libraries

The code utilizes the following external library:

- `secrets:`
 - Used to generate cryptographically strong random numbers suitable for managing data such as passwords.

Functions and Functionalities

- `generate_random_password(length):`
 - Generates a random password of a specified length using characters from the ASCII set (letters, digits, punctuation).
 - Validates the generated password against predefined criteria to ensure it meets specific security requirements.
 - Counts occurrences of different character types (digits, uppercase, lowercase, symbols, repeating characters) in the generated password.
- `calculate_password_strength(password_info):`
 - Computes the strength of a generated password by assigning weights to different password characteristics such as length, character types, and absence of repeating patterns.
 - Calculates a simple strength score based on the provided criteria, indicating the password's strength relative to these factors.

Password Generation and Evaluation

- The `generate_random_password` function creates random passwords based on a specified length.
- Generated passwords undergo validation against predefined criteria to ensure compliance with security standards.
- The `calculate_password_strength` function computes the strength score of the generated password based on its characteristics, providing an overall assessment of its robustness against common security threats.

Password Strength Assessment

- The code outputs a dictionary containing details of the generated password along with its evaluated strength score.
- The strength score is calculated based on the predefined criteria, providing insight into the password's strength relative to the defined checks.

Code #2

```
import secrets
```

```
import string
```

```
import re
```

```
def generate_random_password(length):
```

```
    alphabet = string.ascii_letters + string.digits + string.punctuation
```

```
    password = "".join(secrets.choice(alphabet) for i in range(length))
```

```
    # Check against the specified criteria
```

```
    digit_check = re.search(r"\d", password) is not None
```

```
    uppercase_check = re.search(r"[A-Z]", password) is not None
```

```
    lowercase_check = re.search(r"[a-z]", password) is not None
```

```
    symbol_check = re.search(r"W", password) is not None
```

```
    repeating_chars = re.search(r"(\.)\1", password) is not None
```

```
    birth_years_check = re.search(r"1998|2001|2003", password) is not None
```

```
    # Count occurrences of different types of characters
```

```
    num_digits = len(re.findall(r"\d", password))
```

```
    num_uppercase = len(re.findall(r"[A-Z]", password))
```

```
    num_lowercase = len(re.findall(r"[a-z]", password))
```

```
    num_symbols = len(re.findall(r"W", password))
```

```
    Num_repeating_chars = len(re.findall(r"(\.)\1", password))
```

```
    # Ensure it meets all criteria
```

```
    if (digit_check and uppercase_check and lowercase_check and symbol_check  
        and not repeating_chars and not birth_years_check):
```

```
        password_info = {
```

```
            'Password': password,
```

```
            'NumDigits': num_digits,
```

```
            'NumUppercase': num_uppercase,
```

```
            'NumLowercase': num_lowercase,
```

```
            'NumSymbols': num_symbols,
```

```
            'NumRepeatingChars': num_repeating_chars
```

```
        }
```

```
        strength_evaluation = calculate_password_strength(password_info)
```

```
        password_info['StrengthScore'] = strength_evaluation['StrengthScore']
```

```
        return password_info
```

```
    else:
```

```
        # If the generated password doesn't meet the criteria, regenerate a new one
```

```
        return generate_random_password(length)
```

```

def calculate_password_strength(password_info):
    length = len(password_info['Password'])
    num_digits = password_info['NumDigits']
    num_uppercase = password_info['NumUppercase']
    num_lowercase = password_info['NumLowercase']
    num_symbols = password_info['NumSymbols']
    num_repeating_chars = password_info['NumRepeatingChars']

    # Calculate a simple strength score based on different factors
    strength = length * 4 + (num_digits + num_symbols + num_uppercase + num_lowercase) * 4
    strength -= num_repeating_chars * 2

    return {
        'Password': password_info['Password'],
        'StrengthScore': strength
    }

# Generate a random password of length 12 and evaluate its strength
random_password_info = generate_random_password(12)
print(random_password_info)

```

Code #3 Overview

The provided code assesses the strength and security aspects of passwords. It contains two primary functions and additional functionalities to process and analyze password data:

`check_password(thepassword):`

- Evaluates a given password against predefined criteria, including length, presence of digits, uppercase and lowercase letters, symbols, uniqueness of characters, and common password checks.
- Calculates an overall score based on these criteria to determine the password's strength and security level.

`analyze_from_file(num_entries):`

- Reads a dataset containing password entries (from a file specified by `DATASET_PASSWORDS`) and evaluates each password using the `check_password` function.
- Retrieves and stores the overall score, length, common password status, and count of unique characters for each password entry in a dictionary.

Libraries Used

- `re` (Regular Expressions):
 - Utilized for pattern matching and analysis of password characteristics, such as digit, uppercase, lowercase, and symbol counts, as well as identifying repeating patterns.
- `csv` (CSV File Handling):
 - Used for reading and writing data to CSV files.

Functions and Functionalities

- `check_password(thepassword):`
 - Evaluates a password against specific criteria to determine its strength and security level.
 - Computes various metrics such as length, character types, uniqueness, and checks against common passwords.

- `analyze_from_file(num_entries):`
 - Reads a dataset of password entries from a specified file.
 - Evaluates each password using the `check_password` function and stores relevant metrics for further analysis.

Password Analysis and Data Processing

- The `check_password` function performs comprehensive checks on individual passwords, assessing multiple aspects including length, character types, repetition, uniqueness, and comparison against a list of common passwords.
- The `analyze_from_file` function processes a dataset containing multiple password entries, evaluates each password using `check_password`, and compiles the results into a dictionary containing various metrics for each password.

Output and Data Export

- The code outputs the analyzed password information to the console, displaying the password, its overall score, length, common status, and count of unique characters.
- Additionally, the analyzed password information is exported to a CSV file specified by `output_file_path`, containing columns for password, score, length, common status, and count of unique characters.

Code #3

```
# -*- coding: utf-8 -*-
```

```
import re                # import regex and csv libraries
import csv
```

```
COMMON_PASSWORDS = '10000-most-common-passwords.txt'
DATASET_PASSWORDS = '000webhost.txt'
```

```
def check_password(thepassword):                # password checking function - ex:
    checkpassword("Password123!")
```

```
    pass_length = len(thepassword)
    length_check = pass_length >= 8                # is password 8 characters or more in length?
```

```
    digit_count = len(re.findall(r"\d", thepassword))
    digit_check = re.search(r"\d", thepassword) is not None    # does password have at least one
    digit?
```

```
    uppercase_count = len(re.findall(r"[A-Z]", thepassword))
    uppercase_check = re.search(r"[A-Z]", thepassword) is not None    # does password have at least
    one uppercase character?
```

```
    lowercase_count = len(re.findall(r"[a-z]", thepassword))
    lowercase_check = re.search(r"[a-z]", thepassword) is not None    # does password have at least one
    lowercase character?
```

```
    symbol_count = len(re.findall(r"W", thepassword))
```

```
symbol_check = re.search(r"\W", thepassword) is not None    # does password have at least one
symbol?
```

```
repeat_matches = re.findall(r"(.+?)\1", thepassword)        # does password have repeating
characters?
```

```
repeat_count = len(repeat_matches)
```

```
file = open(COMMON_PASSWORDS, 'r')
```

```
common_passwords = [line.strip() for line in file]
```

```
is_common = thepassword.lower() in common_passwords        # does password match known
common passwords?
```

```
unique_characters = len(set(thepassword))                    # count unique characters in password
```

```
passes_checks = length_check and digit_check and uppercase_check and lowercase_check and
symbol_check and not is_common and not len(repeat_matches) != 0
```

```
length_weight = 1      # add 1 for every character
```

```
digit_weight = 1        # add 1 for every digit
```

```
symbol_weight = 1       # add 1 for every symbol
```

```
repeat_weight = -3      # subtract 3 for every repeat
```

```
common_weight = -10     # subtract 10 if password is common
```

```
overall_score = sum([
    pass_length * length_weight,
    digit_count * digit_weight,
    symbol_count * symbol_weight,
    repeat_count * repeat_weight,
])
```

```
if is_common:
```

```
    overall_score += common_weight
```

```
return {
    'Password Analyzed   ': thepassword,
    'Passes All Checks?  ': passes_checks,
    'At least 8 Length?  ': length_check,
    'At least 1 Digit?   ': digit_check,
    'At least 1 Uppercase?': uppercase_check,
    'At least 1 Lowercase?': lowercase_check,
    'At least 1 Symbol?  ': symbol_check,
    'Repeated Patterns?  ': len(repeat_matches) != 0,
    'Is Common Password?': is_common,
    'Length of Password' : pass_length,
    'Number of Digits    ': digit_count,
    'Number of Uppercase ': uppercase_count,
    'Number of Lowercase ': lowercase_count,
    'Number of Symbols   ': symbol_count,
    'Number of Unique Chars': unique_characters,
```



```

    'Patterns': repeat_matches,
    'Overall Score': overall_score
}

def analyze_from_file(num_entries): # file analyzing function - ex:
    analyze_from_file(700000)
    file = open(DATASET_PASSWORDS, 'r', encoding='utf-8')
    passwords = [line.strip() for line in file][:num_entries]
    password_scores = {}

    for password in passwords: # run the check_password() function on each password
in the file
        result = check_password(password)
        overall_score = result.get('Overall Score') # return overall score
        password_length = result.get('Length of Password') # return password length
        common_pass = result.get('Is Common Password?') # return common check
        unique_chars = result.get('Number of Unique Chars') # return unique chars
        password_scores[password] = {'Score': overall_score, 'Length': password_length, 'Common':
common_pass, 'Unique': unique_chars}

    return password_scores

entries = 1000 # define number of entries to parse
password_scores = analyze_from_file(entries) # run the file analyzing function

for password, score in password_scores.items(): # print results to console
    print(f"Password: {password}, Overall Score: {score['Score']}, Length: {score['Length']}, Common:
{score['Common']}, Unique: {score['Unique']}")

output_file_path = 'output.csv'

with open(output_file_path, 'w', newline="", encoding='utf-8') as output_file:
    csv_writer = csv.writer(output_file)
    csv_writer.writerow(['Password', 'Score', 'Length', 'Common', 'Unique']) # write labels
for columns
    for password, score in password_scores.items():
        csv_writer.writerow([password, score['Score'], score['Length'], score['Common'], score['Unique']])
# write data to row

print(f"Results exported to {output_file_path}") # print success message

```